

Networks and Systems Security II

CSE 354/554

Instructor: Sambuddho

(Semester: Winter 2025)

Week 5: Feb 3 - Feb 7

Brief History of Cryptography

- **Ceaser Cipher / Substitution Cipher**

- “Shift” characters.
- E.g.

PLAINTEXT: ABCDEFGHIJKLMNOPQRSTUVWXYZ

CIPHERTEXT: XYZABCDEFGHIJKLMNOPQRSTUVWXYZ

PLAINTEXT: THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG

CIPHERTEXT: QEB NRFZH YOLTK CLU GRJMP LSBO QEB IXWV ALD

Encryption: $E_n = (x+n) \bmod 26$

Decryption: $D_n = (x-n) \bmod 26$

Brief History of Cryptography

- **Attacking Ceaser Cipher:**

- Simple brute force attacks -- only 26 alphabets in English language.
- Frequency analysis -- E.g. alphabets 'E' (or 'e') and 'T' (or 't') are the most frequent alphabet in English language.
- Encryption of 'E' is always the same...

Brief History of Cryptography

- Medieval Age:

- Vigenere Cipher (Polyalphabetic substitution).
- Inventor: Giovan Battista Bellaso in his 1553 book *La cifra del Sig. Giovan Battista Bellaso* (misattributed to Vigenere).

PLAINTEXT : ATTACKATDAWN

KEY : LEMON



PLAINTEXT : ATTACKATDAWN

KEY : LEMONLEMONL

CIPHER : LXFOPVEFRNHR



KEY : LEMONLEMONL

CIPHER : LXFOPVEFRNHR

PLAINTEXT : ATTACKATDAWN

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
A	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
B	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A
C	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B
D	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C
E	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D
F	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E
G	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F
H	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G
I	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H
J	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I
K	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J
L	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K
M	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L
N	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M
O	O	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N
P	P	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
Q	Q	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
R	R	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
S	S	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
T	T	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S
U	U	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
V	V	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
W	W	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V
X	X	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W
Y	Y	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
Z	Z	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y

Brief History of Cryptography

- **Attacks against Vignere Cipher:**

- Harder to break compared to simple Ceaser substitution.
- Each 'block' is 'effectively' a Ceaser substitution.

Attack strategy:

- Assume the key length is known.
- Assume a cipher stream: `nifon aicum niswt`
- First character 'n', 'a' and 'n' have been encrypted with same key alphabet.
- Second character is encrypted with the same row and so on...
- Perform frequency analysis for each of the first characters. The most frequently occurring character is 'e', the second most frequently occurring is 'a' and so on...
- Perform similar frequency analysis for the other characters as well.

Brief History of Cryptography

- Modern Era (20th Century):

- Electro-mechanical rotor machines to perform multiple substitution cipher.



(Enigma Machine, Image Courtesy: National Cryptographical Museum, NSA, Washington DC)

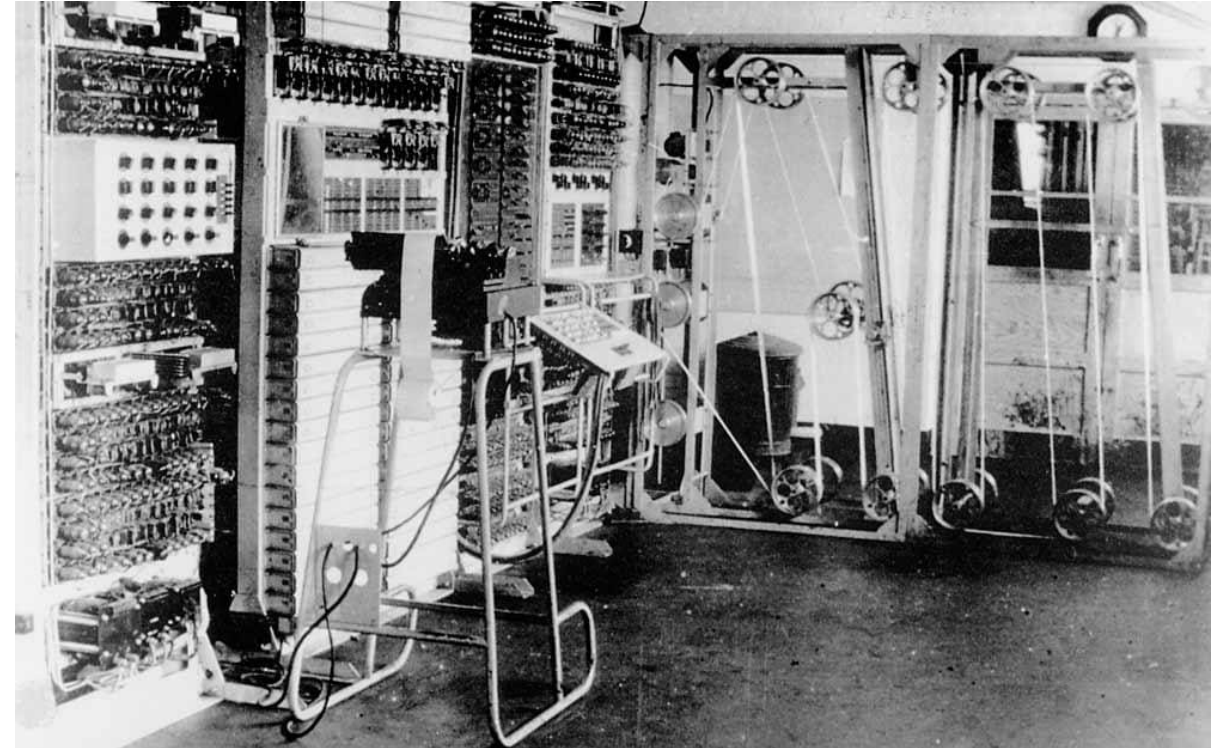
Input: Manual keyboard

Rotor discs: Substitution ciphers

Output: Cipher-stream / telegraphic codes / Lampboards

Plugboard: Additional layer of substitution cipher

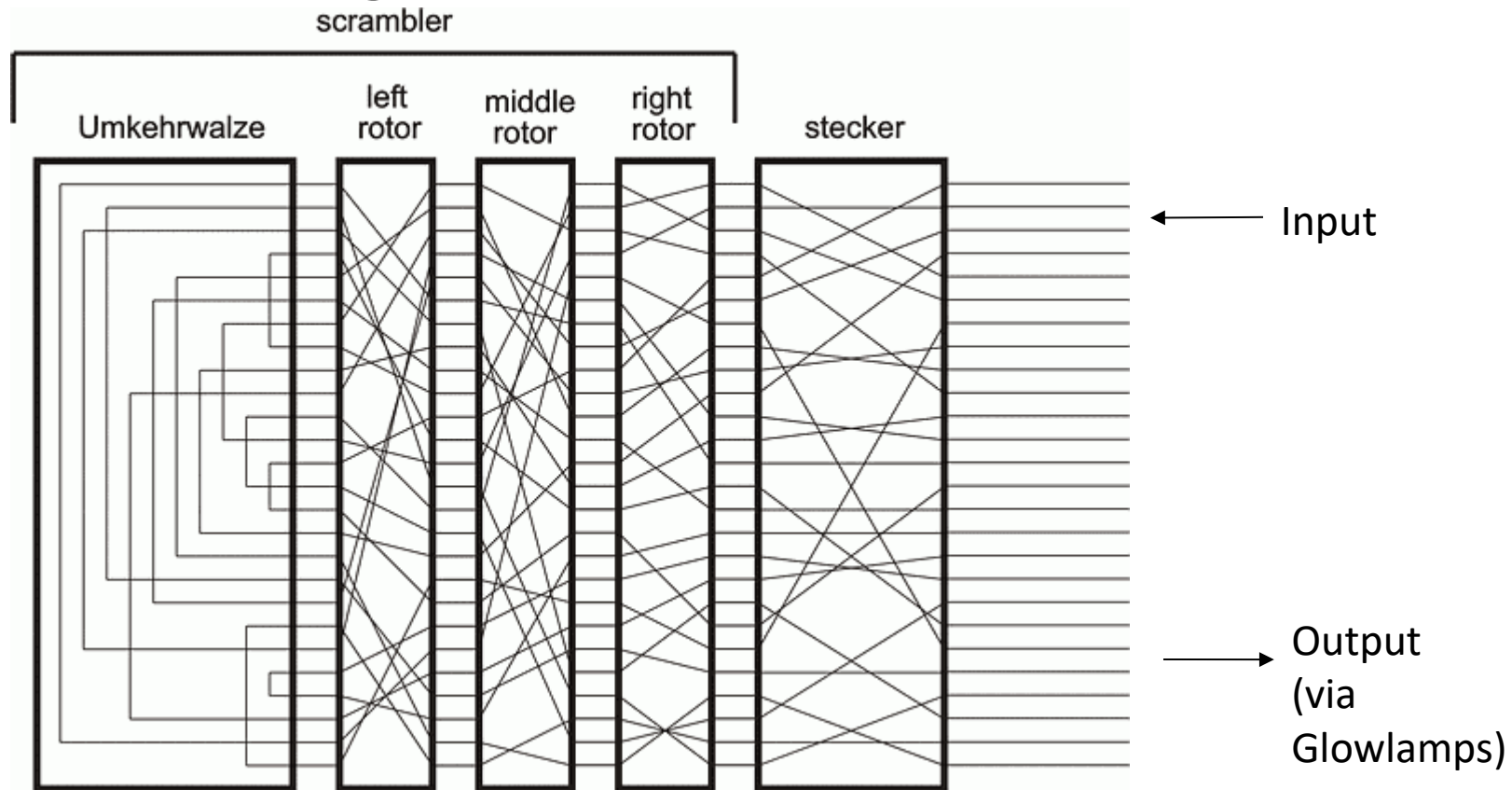
v/s



(Collosus Machine @ Bletchly Park, Image Courtesy: National Cryptographical Museum, NSA, Washington DC)

Brief History of Cryptography

- Inside the Enigma Machine



Some Attack Strategies/Assumptions

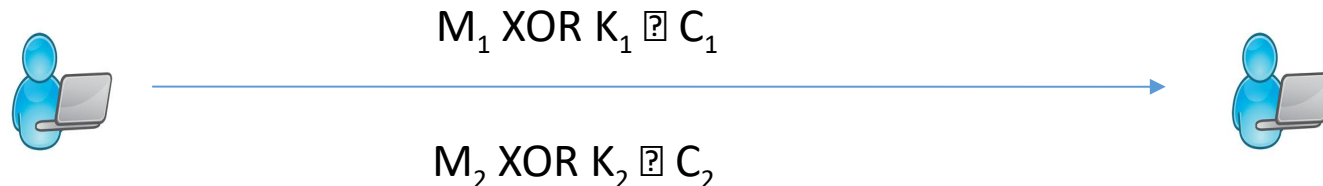
- Known-plaintext attack: The adversary knows plaintext/ciphertext pairs.
- Chosen-plaintext attack: Adversary chooses plaintexts and observes corresponding ciphertexts to which they encrypt.
- Chosen-ciphertext attack: Adversary chooses ciphertexts and sees the corresponding plaintexts to which they decrypt.

Vernam Cipher (One Time Pads)

- Vernam Cipher (One Time Pads (OTPs)) (1917):

PLAINTEXT : H(7) E(4) L(11) L(11) O(14)
KEY : X(23) M(12) C(2) K(10) L(11) + (mod 26)
CIPHER : E(4) Q(16) N(13) V(21) Z(25)

- **Each bit of the plain text is XOR-ed with each bit of the key stream**
- Security lies in length and *pseudo-randomness* of the key stream
- Most basic operation: Message XOR key
- Best use case: Use each key **ONLY** once (ONE TIME...) (considered cryptographically secure...(Shannon – “Perfect Secrecy”)).



$\Pr (E(M_1, K_1)) = \Pr (E(M_2, K_2))$ (for every K_i in K (key space)).
(Cipher text should not reveal the corresponding Message)

Stream Ciphers

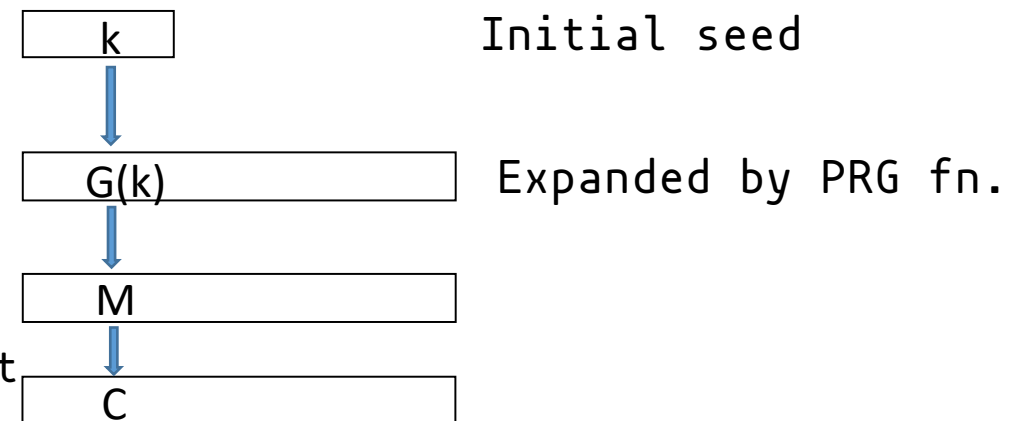
- OTP – Stream Ciphers

- Each key must be used exactly one.
- Multiple usage of same key → statistical cryptanalysis attack
- Reusable keying material
 - Generate new keys without revealing how they are generated.
 - The encrypted cipher text should not reveal any information about the key or the plaintext.
 - Each bit of the message is XOR-ed with the bit of the key.
- Problem with Shannon's "Perfect Secrecy": Key len $\geq$ Message Len
- Solution: PRG instead of perfectly random function.

$$\begin{aligned} E &= G(k) \text{ XOR } M = C \\ D &= G(k) \text{ XOR } C = M \end{aligned}$$

Important property of PRG:

Unpredictability: Knowing some bits $\oplus$ of the key one should not be able to predict the remaining bits.



Some Vulnerabilities of Stream Ciphers

- Multiple time usage of OTP is insecure.

$$C_1 = M_1 \oplus k$$

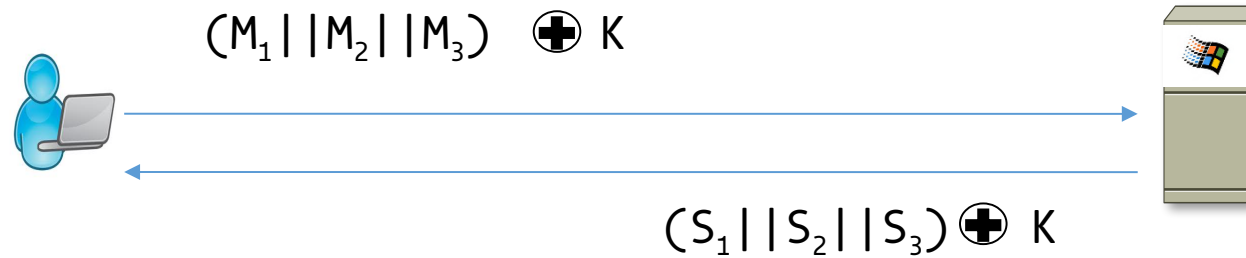
$$C_2 = M_2 \oplus k$$

$$C_1 \oplus C_2 = M_1 \oplus M_2$$

Easy to recover ASCII messages M_1 , M_2 from $M_1 \oplus M_2$

Some Vulnerabilities of Stream Ciphers

- Windows Point-to-Point Tunneling Protocol (MS-PPTP)



- Client and server used the same key.
- Client and server messages could be XOR-ed to reveal an XOR of the client and server messages.
- Other possible attacks - cipher text only attack: The adversary forces the client to generate messages based on specific messages and build an oracle.

Better approach : use different keys

Some Vulnerabilities of Stream Ciphers

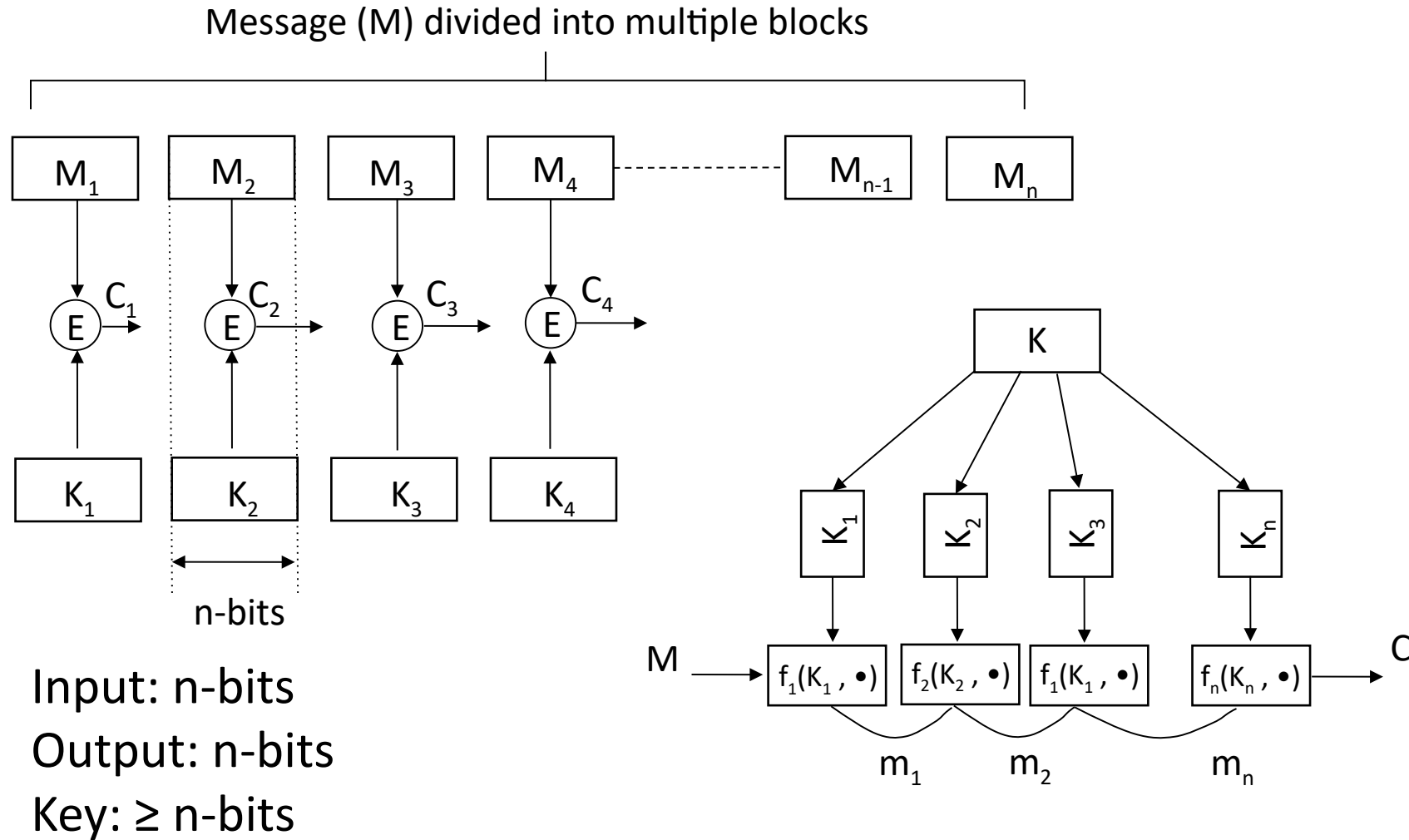
- **Malleability:**

- Adversary can modify the ciphertext without knowing anything about the plaintext and result in equivalent changes to the corresponding plaintext.

Some Popular Examples of Stream-Ciphers

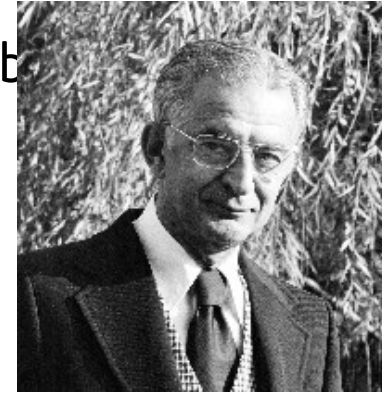
- RC4:
 - Date of creation: 1987
 - Key length: 8-bits (seeds) - 2048-bits (generator)
 - Randomization: None, if IV used then key must be mixed.
 - Remarks: HTTPS, WEP (broken)
 - Attack: Chosen plain text attack.
- A5/1, A5/2:
 - Date of creation: 1989
 - Key length: 54-bits (2G)
 - Randomization: 114-bits
 - Remarks: Voice encryption for GSM networks.
 - Attack: Chosen plain text attack.
- Salsa:
 - Date of creation: 2004
 - Key length: 256-bits; cipher stream length: 512-bits.
 - Randomization: 64-bit nonce.
 - Remarks: Optimized for hardware implementations
 - Attack: Aumasson, Fischer, Khazaei, Meier, and Rechberger, 2008. - *probabilistic neutral bits attack*.

Block Ciphers



Brief History of Block Ciphers

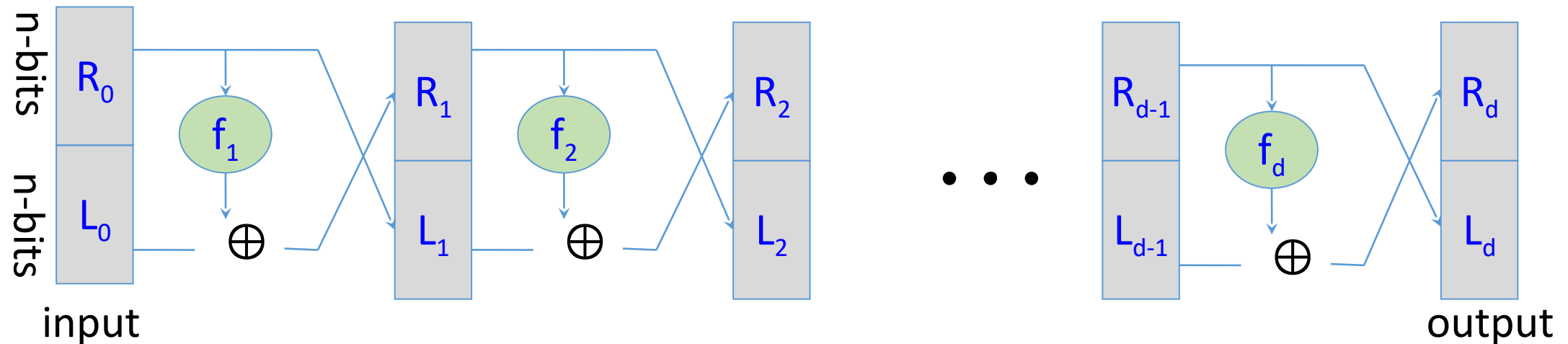
- Early 1970s: Horst-Feistel proposed Lucifer block cipher.
 - Block size : 128-bits, Key: 128-bits
- 1973: NBS (now NIST) asked for block cipher proposals.
 - IBM submits Lucifer.
- 1976: NBS adopts Lucifer with shorter key length and it is called Data Encryption Standards (DES).
 - Block size: 64-bits key-len: 56-bits
- 1997: DES broken by exhaustive search (brute force search).
- 2000: NIST adopts Rijndael as AES and replaces DES.



DES: core idea - Feistel Network

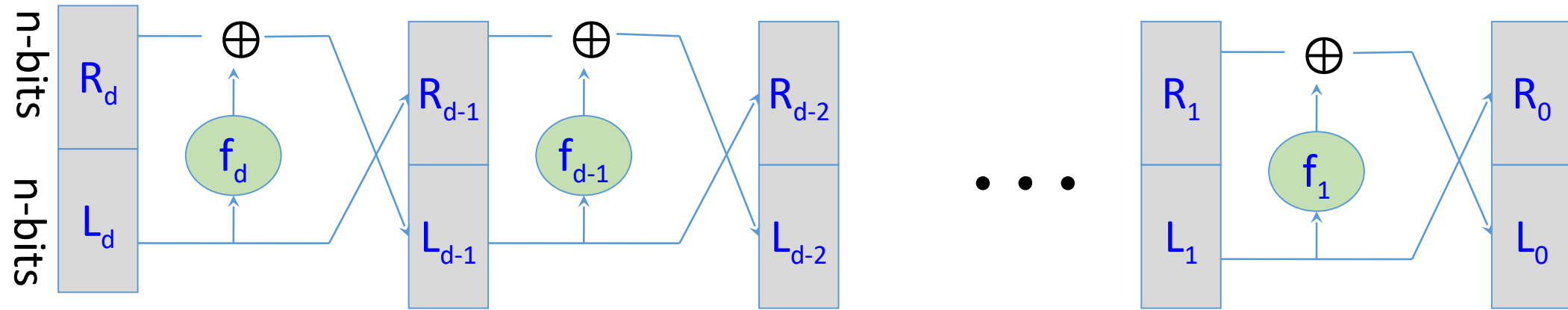
Given functions $f_1, \dots, f_d: \{0,1\}^n \rightarrow \{0,1\}^n$

Goal: build invertible function $F: \{0,1\}^{2n} \rightarrow \{0,1\}^{2n}$



In symbols: $R_i = F_i(R_{i-1}) \oplus L_i$
 $L_i = R_{i-1}$

Decryption circuit



- Inversion is basically the same circuit,
with $f_1, \dots, f_d$ applied in reverse order
- General method for building invertible functions (block ciphers)
from arbitrary functions.
- Used in many block ciphers ... but not AES

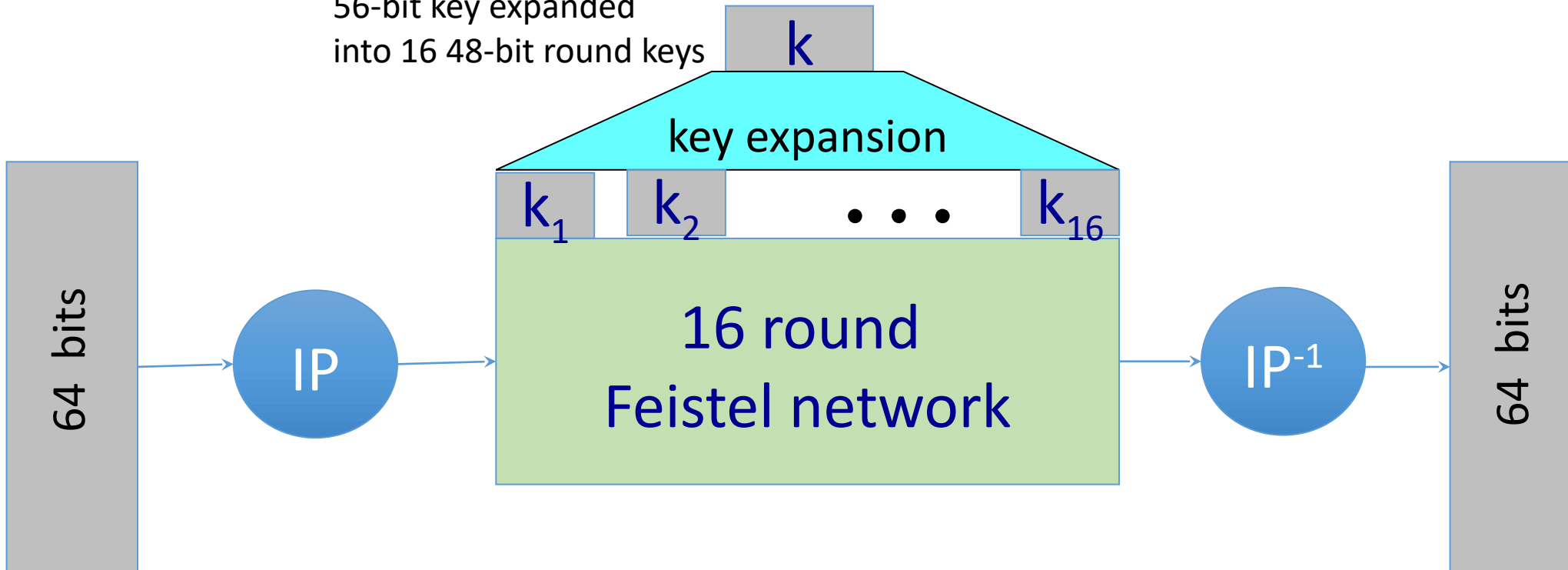
DES: 16 round Feistel network

$$f_1, \dots, f_{16}: \{0,1\}^{32} \rightarrow \{0,1\}^{32}, \quad f_i(x) = F(k_i, x)$$

derived

(Round key
from key K)

56-bit key expanded
into 16 48-bit round keys



To invert, use keys in reverse order

output

DES challenge

msg = "The unknown messages is: XXXX ..."
CT = c_1 c_2 c_3 c_4

Goal: find $k \in \{0,1\}^{56}$ s.t. $\text{DES}(k, m_i) = c_i$ for $i=1,2,3$

1997: Internet search -- 3 months

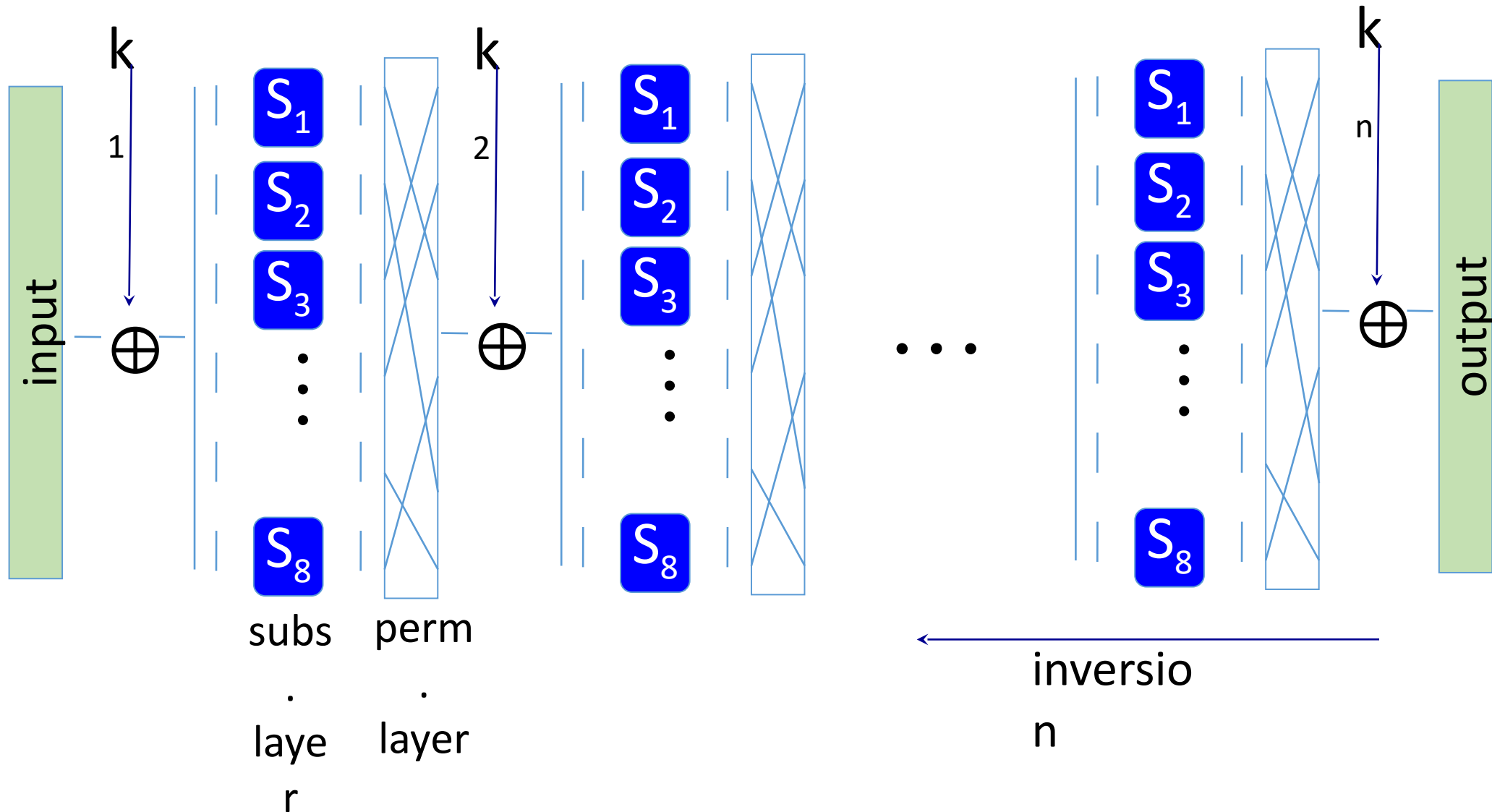
1998: EFF machine (deep crack) -- 3 days (250K \$)

1999: combined search -- 22 hours

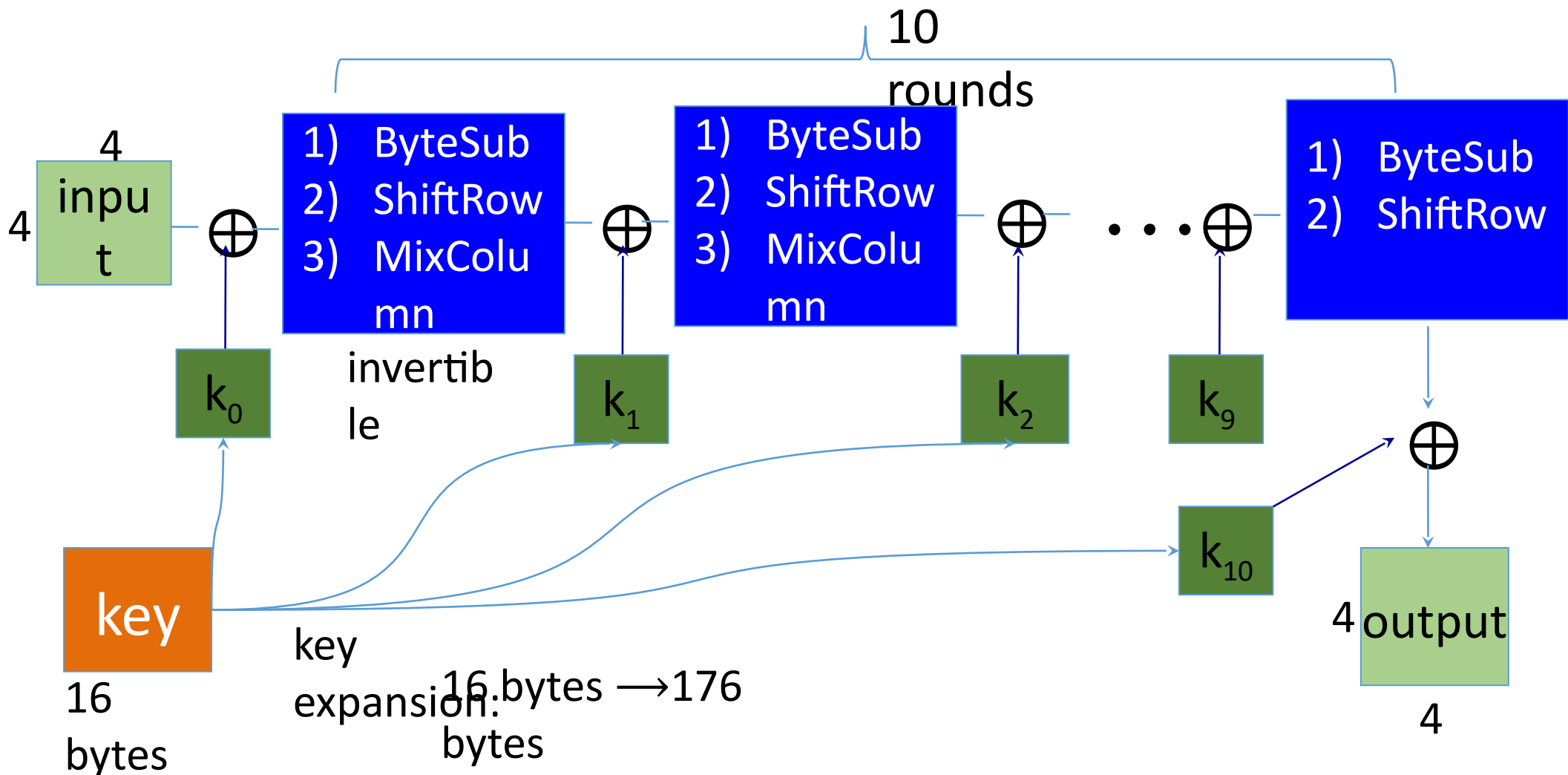
2006: COPACOBANA (120 FPGAs) -- 7 days (10K \$)

$\Rightarrow$ 56-bit ciphers should not be used !! (128-bit key $\Rightarrow 2^{72}$ days)

AES: Advanced Encryption Standard Sub-Perm network (not Feistel)



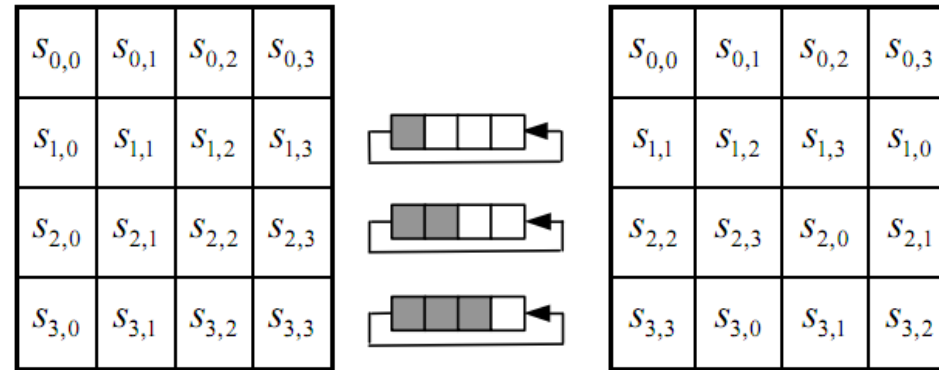
AES-128 schematic



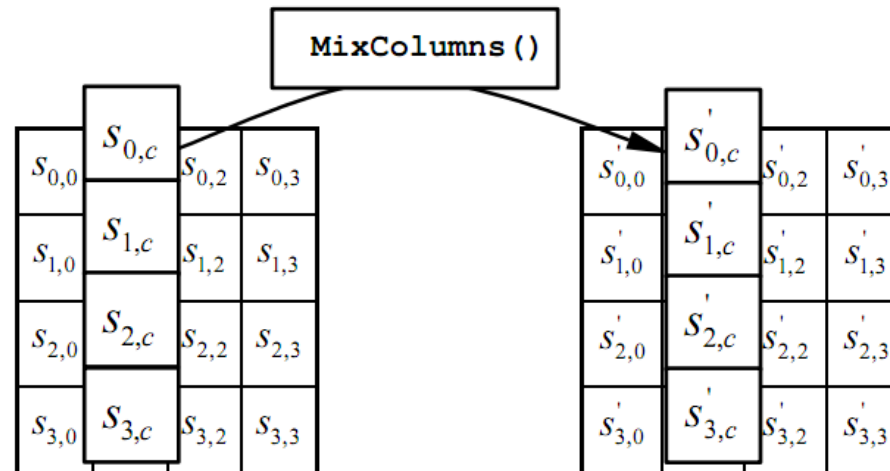
The round function

- **ByteSub:** a 1 byte S-box. 256 byte table (easily computable)

- **ShiftRows:**



- **MixColumns:**



Code size/performance tradeoff

	Code size	Performance
Pre-compute round functions (24KB or 4KB)	largest	fastest: table lookups and XORs
Pre-compute S-box only (256 bytes)	smaller	slower
No pre-computation	smallest	slowest

AES in hardware

AES instructions in Intel Westmere:

- `aesenc`, `aesenclast`: do one round of AES
128-bit registers: `xmm1=state`, `xmm2=round key`
`aesenc xmm1, xmm2` ; puts result in `xmm1`
- `aeskeygenassist`: performs AES key expansion
- Claim 14 x speed-up over OpenSSL on same hardware

Similar instructions on AMD Bulldozer

Semantic Security for many-time key

Key used more than once $\Rightarrow$ adv. sees many CTs with same key

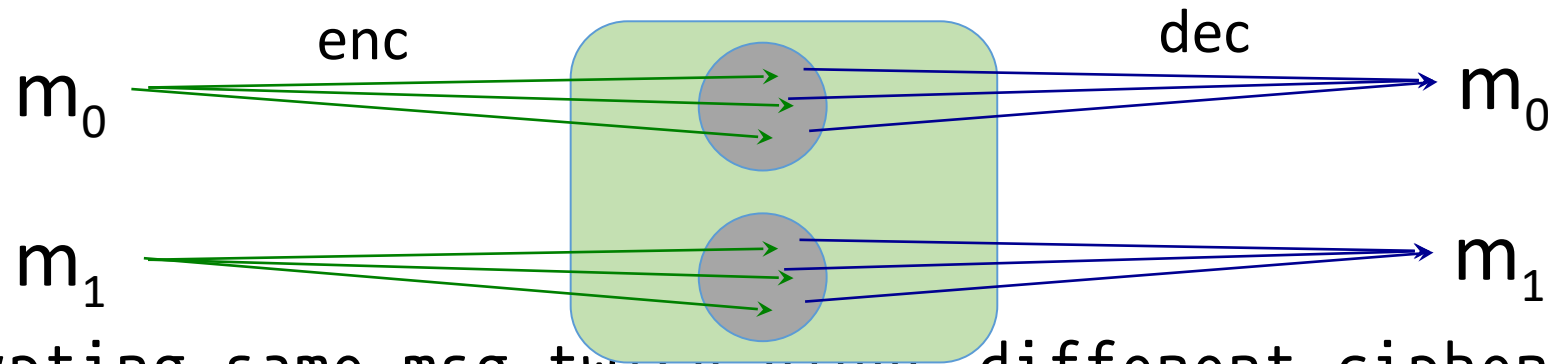
Adversary's power: chosen-plaintext attack (CPA)

- Can obtain the encryption of arbitrary messages of his choice
(conservative modeling of real life)

Adversary's goal: Break semantic security

Solution 1: randomized encryption

- $E(k,m)$ is a randomized algorithm:

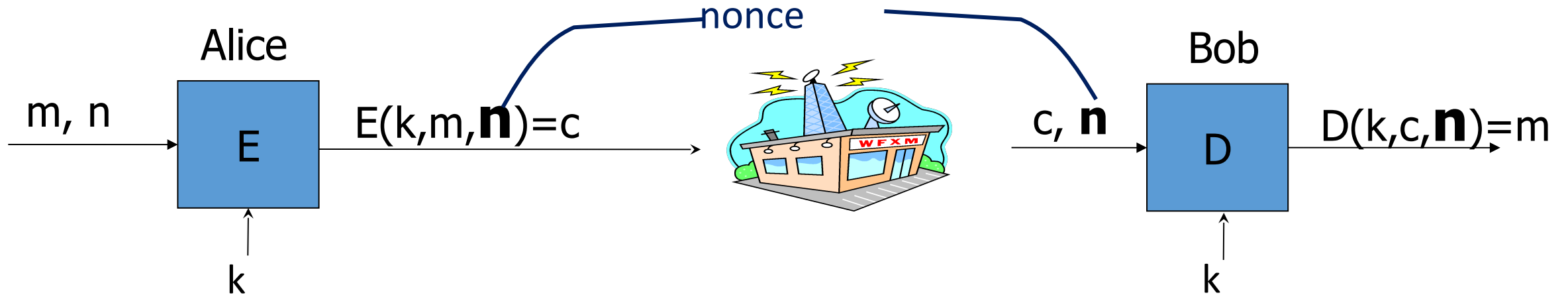


⇒ encrypting same msg twice gives different ciphertexts (w.h.p)

⇒ ciphertext must be longer than plaintext

Roughly speaking: CT-size = PT-size + “# random bits”

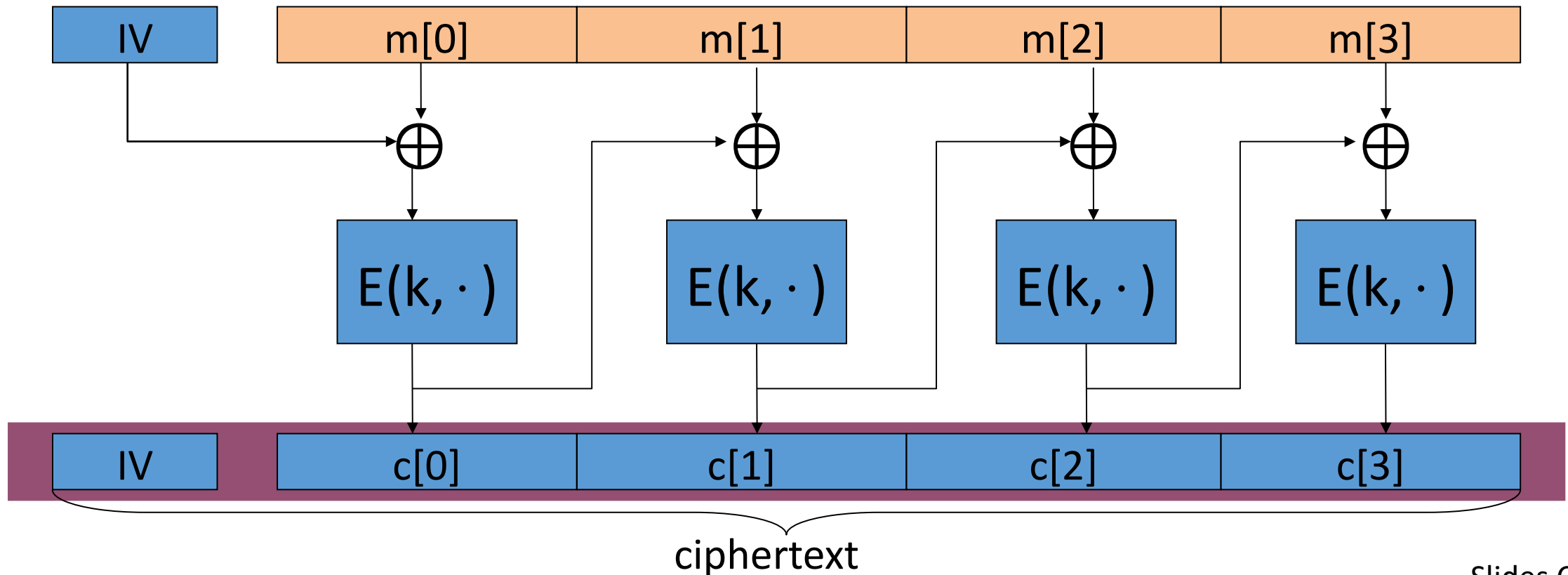
Solution 2: nonce-based Encryption



- nonce n : a value that changes from msg to msg.
(k, n) pair never used more than once
- method 1: nonce is a counter (e.g. packet counter)
 - used when encryptor keeps state from msg to msg
 - if decryptor has same state, need not send nonce with CT
- method 2: encryptor chooses a random nonce, $n \leftarrow N$

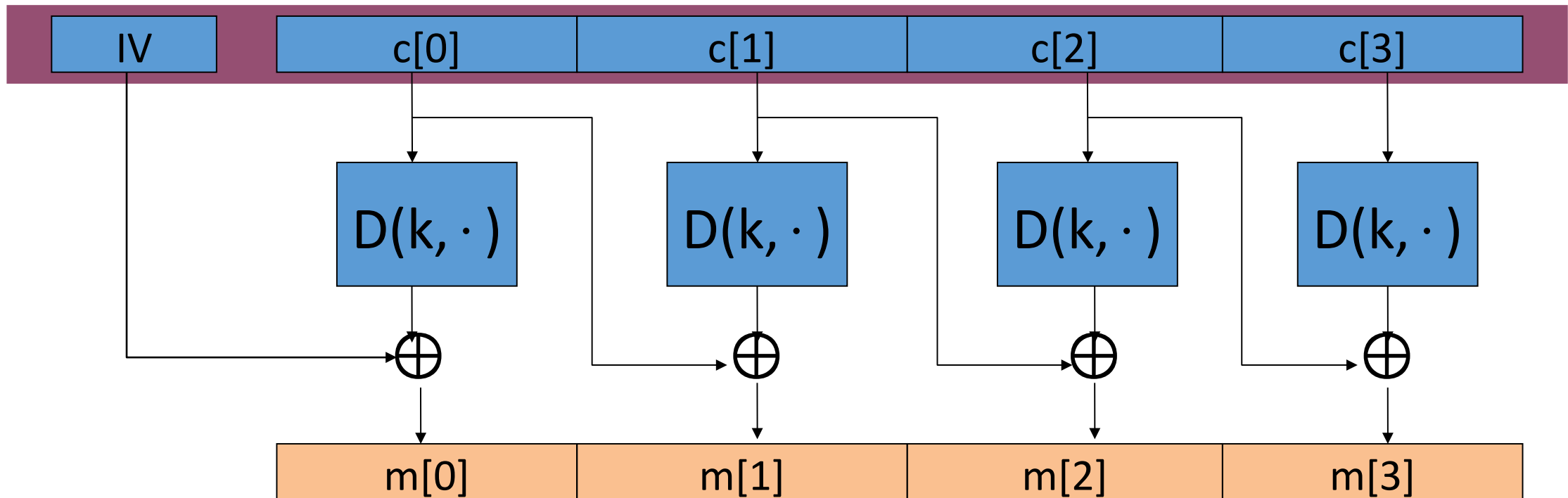
Construction 1: CBC with random IV

$E_{\text{CBC}}(k, m)$: choose random IV:



Decryption circuit

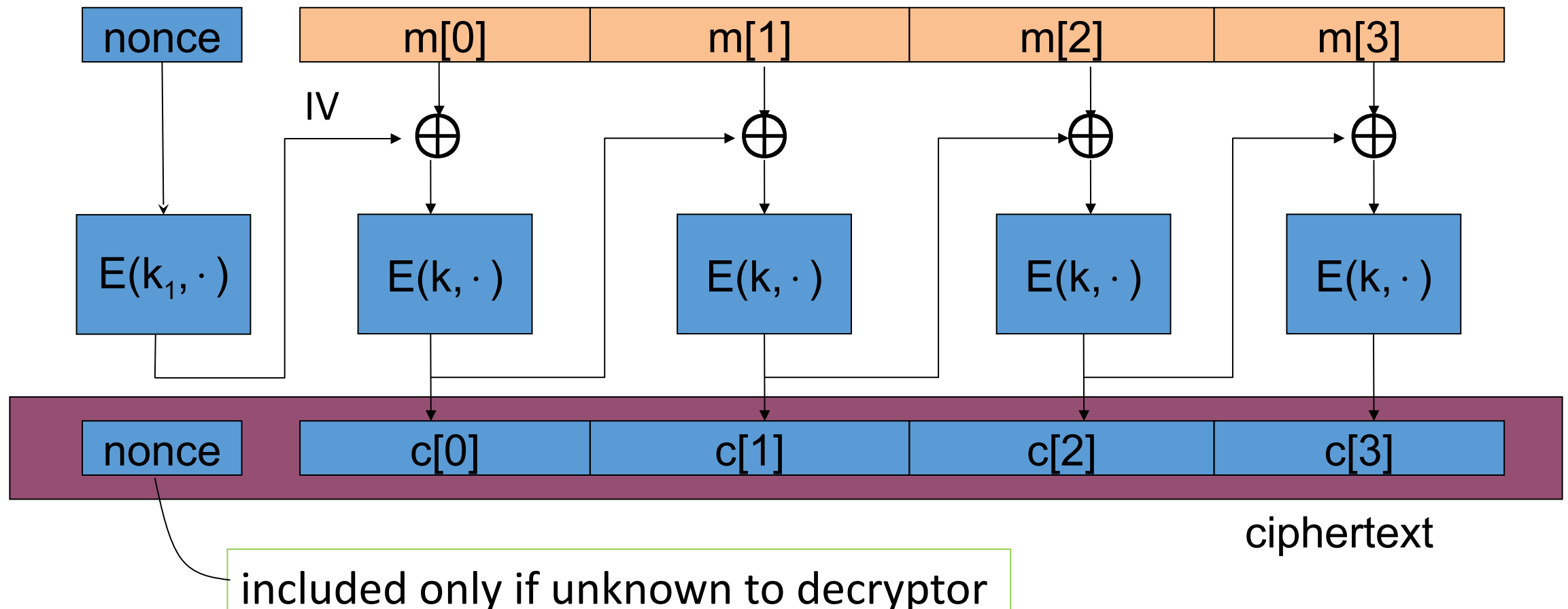
In symbols: $c[0] = E(k, IV \oplus m[0]) \Rightarrow m[0] = D(k, c[0]) \oplus IV$



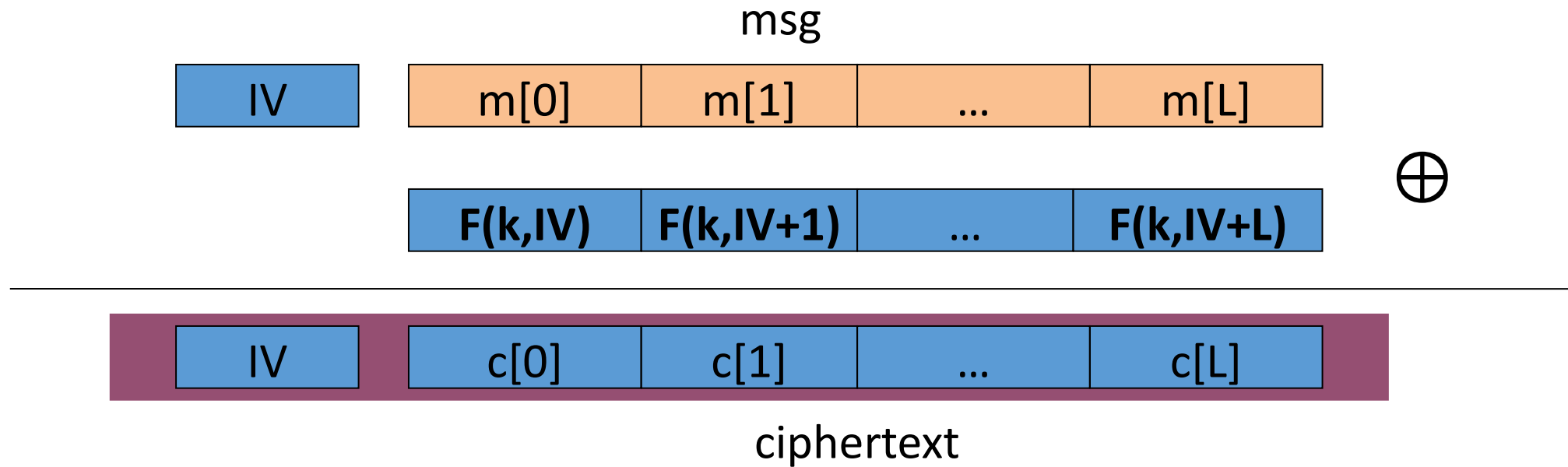
Construction 2: Nonce-based CBC

- Cipher block chaining with unique nonce: $\text{key} = (k, k_1)$

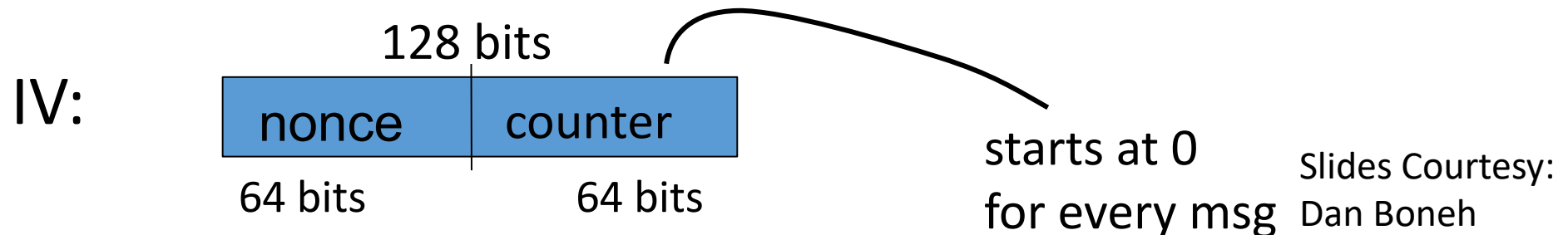
unique nonce means: (key, n) pair is used for only one message



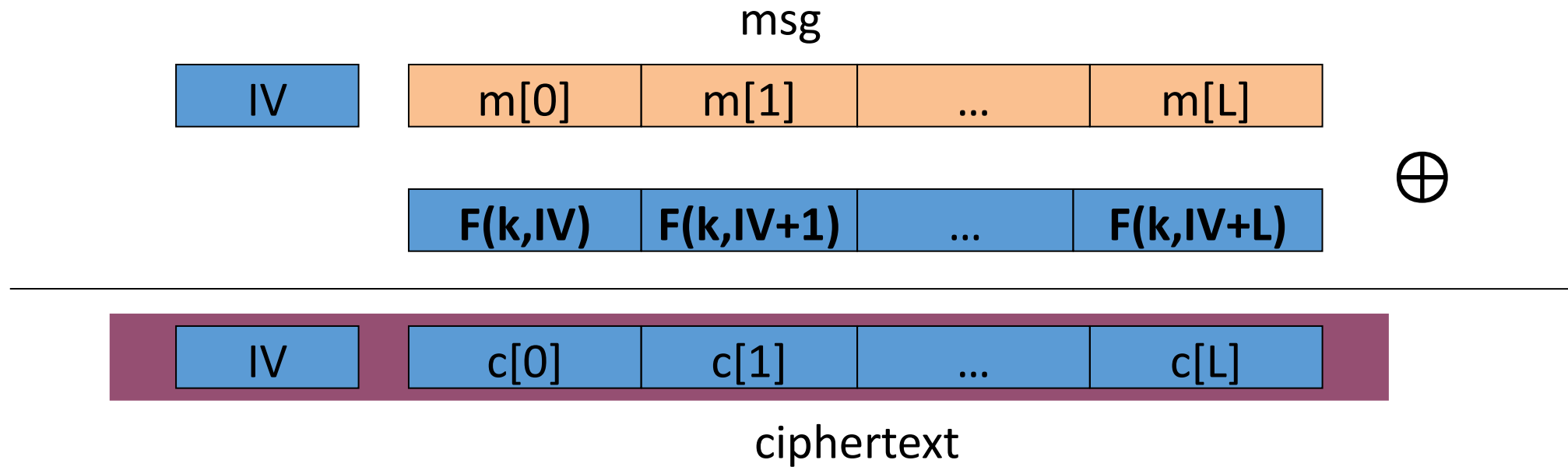
Construction 4: nonce ctr-mode



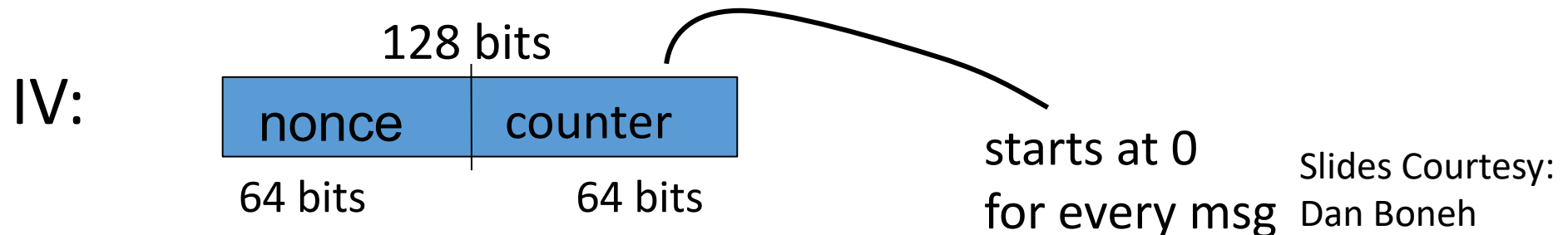
To ensure $F(k, x)$ is never used more than once, choose IV as:



Construction 4: nonce ctr-mode



To ensure $F(k,x)$ is never used more than once, choose IV as:



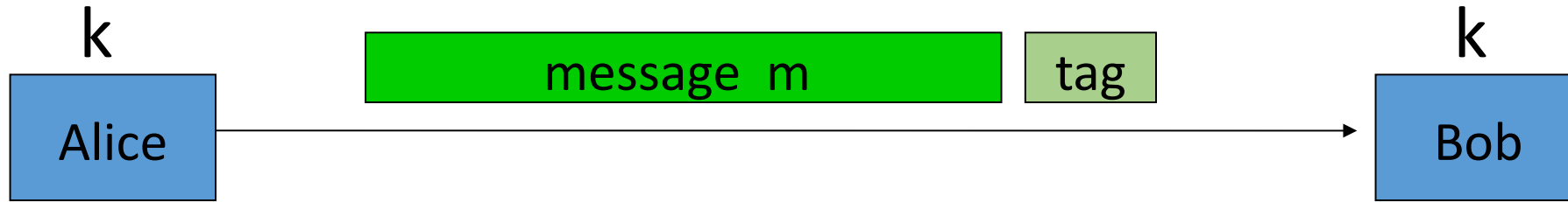
Message Integrity/Message Authentication

Goal: integrity, no confidentiality.

Examples:

- Protecting public binaries on disk.
- Protecting banner ads on web pages.

Message integrity: MACs



Generate tag:

$$\text{tag} \leftarrow S(k, m)$$

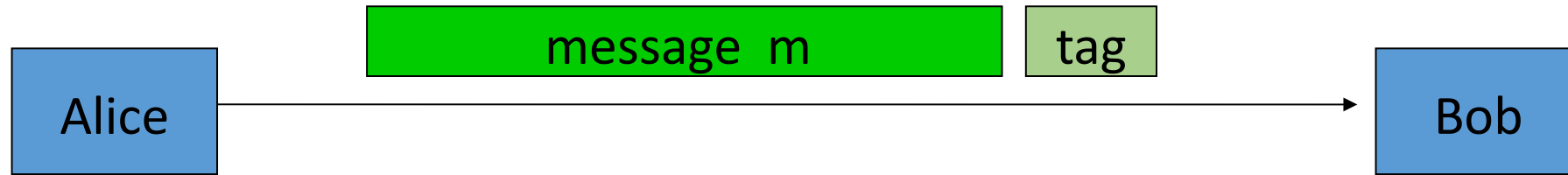
Verify tag:

$$V(k, m, \text{tag}) \stackrel{?}{=} \text{'yes'}$$

Def: MAC $I = (S, V)$ defined over (K, M, T) is a pair of algs:

- $S(k, m)$ outputs t in T
- $V(k, m, t)$ outputs 'yes' or 'no'

Integrity requires a secret key



Generate tag:

$\text{tag} \leftarrow \text{CRC}(m)$

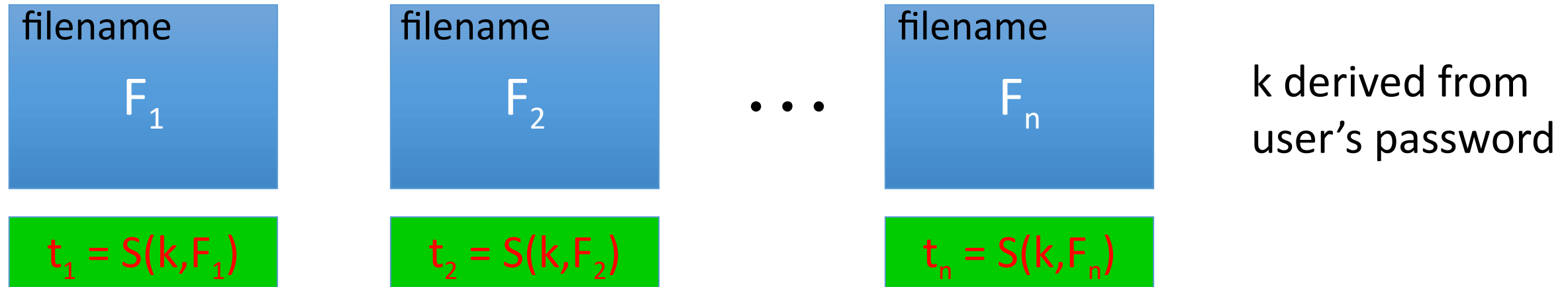
Verify tag:

$V(m, \text{tag}) \stackrel{?}{=} \text{'yes'}$

- Attacker can easily modify message m and re-compute CRC.
- CRC designed to detect random, not malicious errors.

Example: protecting system files

Suppose at install time the system computes:



Later a malware infects system and modifies system files

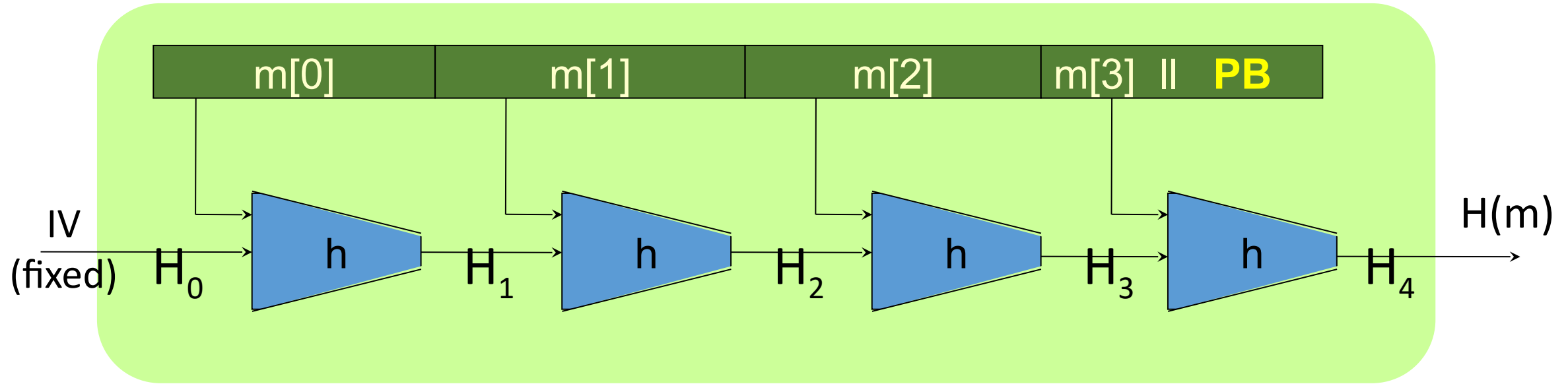
User reboots into clean OS and supplies his password

- Then: secure MAC $\Rightarrow$ all modified files will be detected

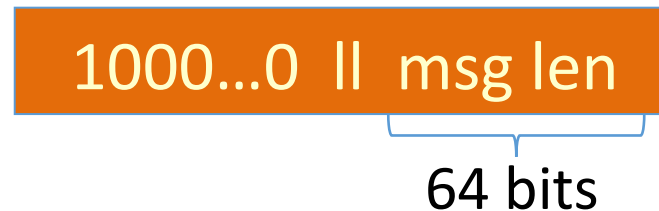
Attacks Against MACs

- *Existential Forgery*: The attacker produces a signature s of some message m of his choice.
- *Selective Forgery*: The attacker chooses a message m , then gets access to the public key used for verification, and produces a signature s of m .
- *Key recovery*: Given the public key for verification, the attacker produces the secret key for signing.

The Merkle-Damgard Iterated Construction

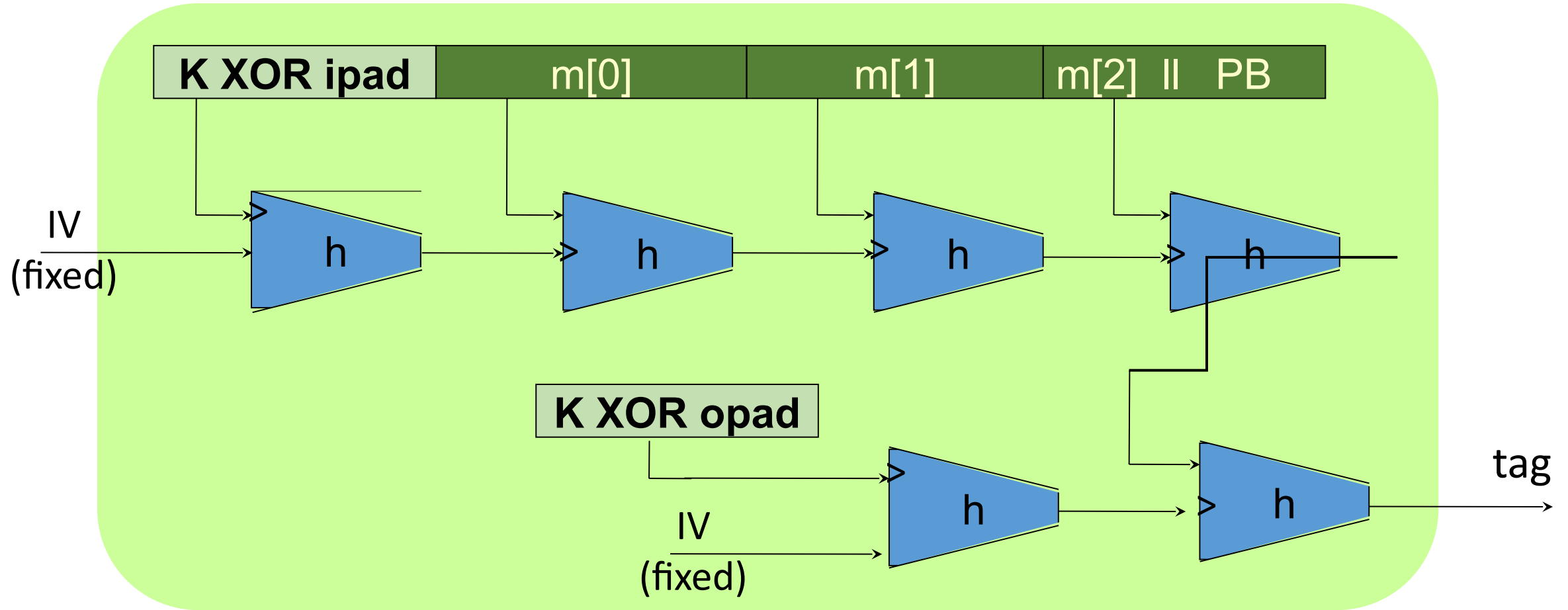


PB: padding block

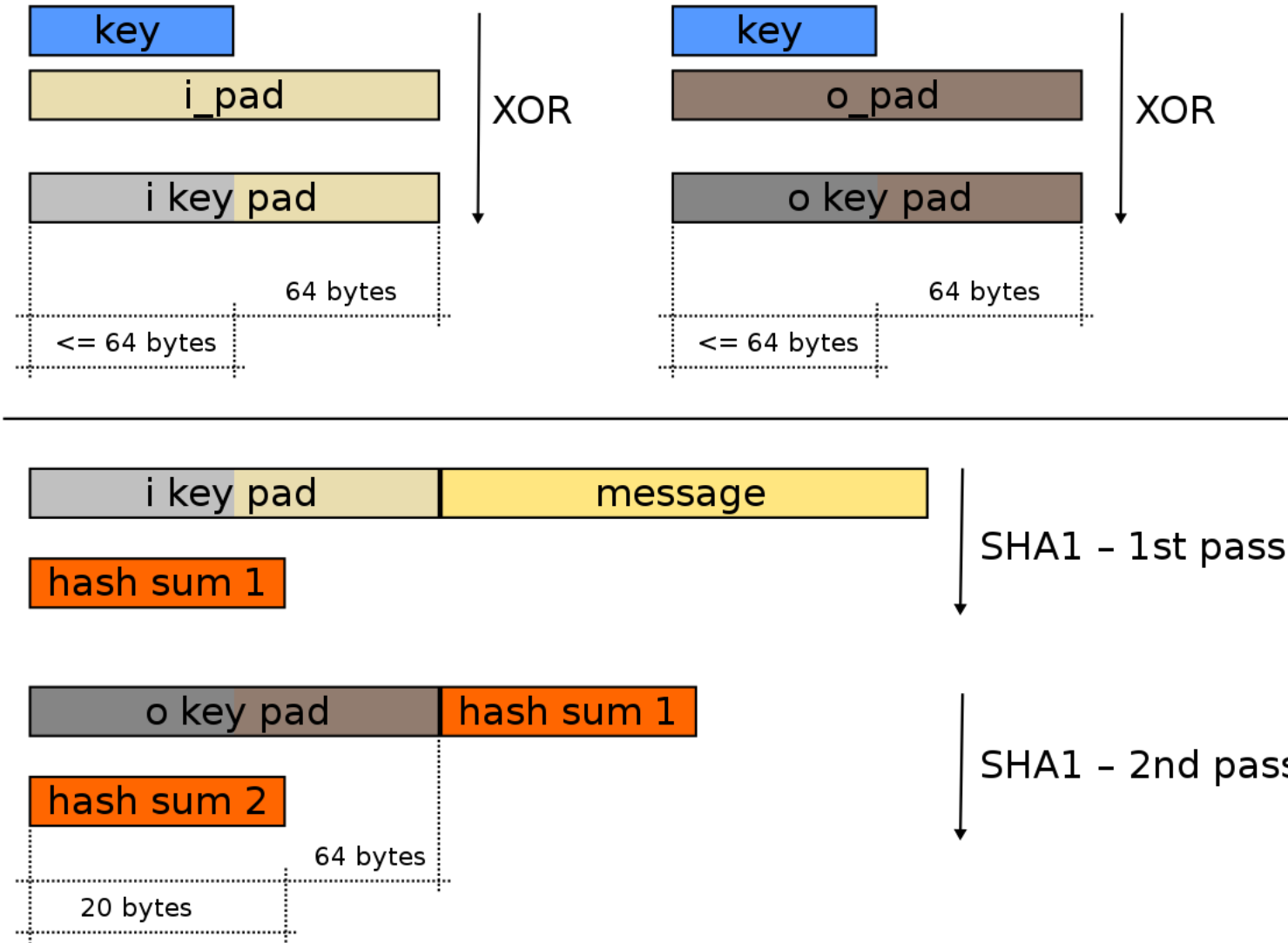


If no space for PB
add another block

HMAC in pictures



HMAC in pictures



Standardized method: HMAC (Hash-MAC)

Most widely used MAC on the Internet.

Use for calculating a message authentication code (MAC) involving a cryptographic hash function that may use a secret cryptographic key.

H: hash function.

example: SHA-256 ; output is 256 bits

Building a MAC out of a hash function:

HMAC: $S(k, m) = H(k \oplus \text{opad} \parallel H(k \oplus \text{ipad} \parallel m))$

ipad : chosen value repeated to match keylen $\rightarrow 0x36$

opad : chosen value repeated to match keylen $\rightarrow 0x5c$

AES Galios/Counter Mode (GCM)

Authenticated Encryption with Associated Data (AEAD)

Provides an “intrinsic” authentication tag.
Efficient – well suited for H/w implementation

GHASH: Galios (Keyed)
Hash

